Payment Service

This service is responsible to process credit card payments for orders. It will return an error if the credit card is invalid or the

Zero-code instrumentation

This Node.js based service makes use of the OpenTelemetry Node.js Zero-code Instrumentation, setting up by requiring the `@opentelemetry/auto-instrumentations-node/register` module at startup. Export endpoints, resource attributes, and service name are automatically set based on environment variables. This can be done in the service's `package.json` start script or via `NODE_OPTIONS`.

```
"scripts": {
  "start": "node --require @opentelem
}
```

Traces

Add attributes to auto-instrumented spans

Within the execution of auto-instrumented code you can get current span from context.

```
const span = opentelemetry.trace.getA
```

Adding attributes to a span is accomplished using `setAttributes` on the span object. In the `chargeServiceHandler` function an attributes is added to the span as an anonymous object (map) for the attribute key/values pair.

```
span?.setAttributes({
  'demo.payment.amount': parseFloat(
    2,
  ),
});
```

Span Exceptions and status

You can use the span object's `recordException` function to create a span event with the full stack trace of a handled error. When recording an exception also be sure to set the span's status accordingly. You can see this in the `charge` function in `charge.js`.

```
span.recordException(err);
span.setStatus({ code: SpanStatusCode
```

Metrics

Creating Meters and Instruments

Meters can be created using the `@opentelemetry/api` package. You can create meters as seen below, and then use the created meter to create instruments.

```
const { metrics } = require('@opente

const meter = metrics.getMeter('payme
const transactionsCounter = meter.cre
```

Meters and Instruments are supposed to stick around. This means you should get a Meter or an Instrument once , and then re-use it as needed, if possible.

Logs

TBD

Baggage

OpenTelemetry Baggage is leveraged in this

service to check if the request is synthetic (from the load generator). Synthetic requests will not be charged, which is indicated with a span attribute. The `charge.js` file which does the actual payment processing, has logic to check the baggage.

```
// check baggage for synthetic_reques
const baggage = propagation.getBaggag
if (
  baggage &&
  baggage.getEntry('synthetic_request
  baggage.getEntry('synthetic_request
) {
  span.setAttribute('demo.payment.cha
} else {
  span.setAttribute('demo.payment.cha
}
```

Last modified June 24, 2026: [docs: update payment service for zero-code instrumentation (#10511) (faacefc5)]

Ask AI